ASSIGNMENT 4 – TASK 6

CAMPUS EVENT REGISTRATION DESKTOP APPLICATION

Course: Programming in Java | CO4 | Bloom's Level: K5 – Evaluate | 13 Marks

1. Stage 1 – Interpret the problem

Users: students register for events; committee members view, filter and export registrations. **Data:** student ID, student name, email, event name, registration ID and registration timestamp. **Rules:** required fields cannot be blank; email must have a valid basic format; a student may register only once for a given event; the table supports event filtering and paging; export produces a CSV; database work must not block the Swing Event Dispatch Thread.

ID	Testable requirement
R1	A student can submit student ID, name, email and one selected event.
R2	If any required field is blank, the app rejects the request and shows a clear error.
R3	If email syntax is invalid, the app rejects the request before insertion.
R4	The database rejects a second row with the same student ID and event.
R5	Committee users can view registrations in a JTable and filter by event.
R6	The table loads at most 50 records per page using LIMIT and OFFSET.
R7	The current event filter can be exported as a CSV file.
R8	Registration, page loading and export execute in SwingWorker background tasks.
R9	If the user closes the window during an active operation, the app asks for confirmation.

2. Stage 2 – Screen and event flow

Main window: JFrame with JMenuBar (File → Export CSV, Refresh, Exit); registration JPanel containing Student ID, Student Name, Email and Event JComboBox plus Register button; central JTable inside JScrollPane; bottom filter JComboBox, Previous/Next controls and status label. Dialogs communicate successful registration, invalid input, duplicate registration, export completion and close confirmation.

```
JFrame EventRegistrationApp
■■■ JMenuBar
■   ■■■ File: Export CSV | Refresh | Exit
■■■ Registration JPanel (BorderLayout)
■   ■■■ Form JPanel (GridLayout 2x4)
■   ■   ■■■ JTextField: studentId, studentName, email
■   ■   ■■■ JComboBox: eventChoice
■   ■■■ JButton: Register
■■■ JScrollPane
■   ■■■ JTable (registration rows)
■■■ Bottom JPanel
    ■■■ JComboBox: filterChoice
    ■■■ JButton: Previous / Next
    ■■■ JLabel: status

Register → validate → SwingWorker → Service → Repository → MySQL
Filter/Refresh → SwingWorker → SQL COUNT + LIMIT/OFFSET → JTable
Export → JFileChooser → SwingWorker → CSV file
```

3. Stage 3 – Critical design decisions

Validation: RegistrationService checks required values and basic email syntax before calling the repository. UI fields are collected on the EDT; validation and database work are performed in the worker task. **Duplicate prevention:** the database has UNIQUE(student_id, event_name). This is authoritative and prevents race conditions that a preliminary SELECT-only check cannot prevent. SQL uses PreparedStatement to avoid concatenating user input. **Responsiveness:** registration, count/page retrieval and CSV export run inside SwingWorker.doInBackground(); SwingWorker.done() returns to the EDT to update controls and show dialogs. Connections and statements use try-with-resources.

4. Stage 4 – Class and handler design

```
EventRegistrationApp (JFrame)
   | calls
   v
```

```
RegistrationService (validation + rules)
   | calls
   v
RegistrationRepository (JDBC queries / CSV export)
   | uses
   v
Database (connection configuration) → MySQL

Registration (data model) → rows displayed in JTable
```

Register pseudocode

```
INPUT studentId, studentName, email, eventName
IF any required field is empty THEN show validation error; RETURN
IF email format is invalid THEN show validation error; RETURN
START SwingWorker
    OPEN database connection
    INSERT registration using PreparedStatement
    IF unique-key violation THEN report duplicate registration
ON worker completion (EDT)
    IF success THEN show confirmation; clear fields; reload first page
    ELSE show clear error dialog
```

Filter pseudocode

```
READ selected event filter
RESET page index to 0
START SwingWorker
    total ← SELECT COUNT(*) with optional event predicate
    rows ← SELECT ... ORDER BY ... LIMIT 50 OFFSET page*50
ON worker completion (EDT)
    replace JTable model rows
    enable/disable Previous and Next
    update status with page and total count
```

5. Stage 5 – Implementation and demonstration

The supplied source package implements the layered Swing application. It includes a registration form, menu bar, JTable, event filter, 50-row page navigation, CSV export, service validation, prepared SQL statements, a database uniqueness constraint and SwingWorker background operations. The actual database demonstration requires MySQL Server, the schema script, Connector/J and the user's local credentials; those cannot be verified from this report-generation environment.

6. Database design

```
registrations(
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  student_id VARCHAR(30) NOT NULL,
  student_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  event_name VARCHAR(100) NOT NULL,
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(student_id, event_name)
)
```

An index on (event_name, registered_at, id) supports event-filtered ordering. The unique constraint is the final duplicate-registration guard.

7. Alternative analysis – load all rows versus LIMIT/OFFSET

For 100,000 registrations, let R be the average in-memory row footprint including object, strings and table-model overhead. The exact R depends on JVM, string lengths and table implementation; for a transparent planning estimate use R = 1 KiB per row. Loading all rows therefore needs approximately 100,000 × 1 KiB = 97.7 MiB for row data alone, excluding driver buffers and temporary objects. A page size of 50 needs about 50 KiB of row data in the table model, approximately 2,000 times less row-model memory. The database still stores all 100,000 rows in both approaches.

Measure	Load all 100,000 rows	50-row LIMIT/OFFSET
Rows transferred per page	100,000	At most 50
Client table row memory (assumption)	About 97.7 MiB at 1 KiB/row	About 50 KiB at 1 KiB/row
Initial data transfer	O(N) rows	O(P) rows, P=50
Window responsiveness	Large fetch/model update can freeze EDT if not moved to a worker	Smaller fetch keeps initial transfer and model update; background worker keeps E
Deep-page query	No paging query	OFFSET may scan/skip many earlier rows; keyset pagination may scale bette

Let N=100,000 and P=50. Fetching all rows is O(N) transfer and O(N) client row storage. A page query returns O(P) rows and the visible model stores O(P) rows. A conventional OFFSET query can still require the database to walk past O(offset + P) qualifying rows, so later pages may become slower. An indexed keyset/cursor approach can reduce this cost but requires changing the

paging interface. SwingWorker is important in both approaches: it prevents JDBC and table preparation from running on the EDT, but it cannot eliminate the memory and transfer cost of loading all rows.

8. Test plan and evidence

Execute the application against the local MySQL database and capture genuine screenshots: (1) successful registration confirmation; (2) invalid input message; (3) duplicate registration rejected; (4) filtered JTable; (5) CSV export result. Include the CSV file and screenshots in Task06. Test duplicate protection by submitting the same student ID and event twice. Do not claim a database test passed until it has been run against the configured server.

9. Conclusion

The design separates Swing presentation, application validation and JDBC persistence. Prepared statements protect SQL inputs; a database unique constraint enforces one registration per student per event; SwingWorker keeps long-running operations off the EDT; and paginated table loading limits client memory and initial transfer. The supplied implementation is ready for local MySQL configuration and execution.